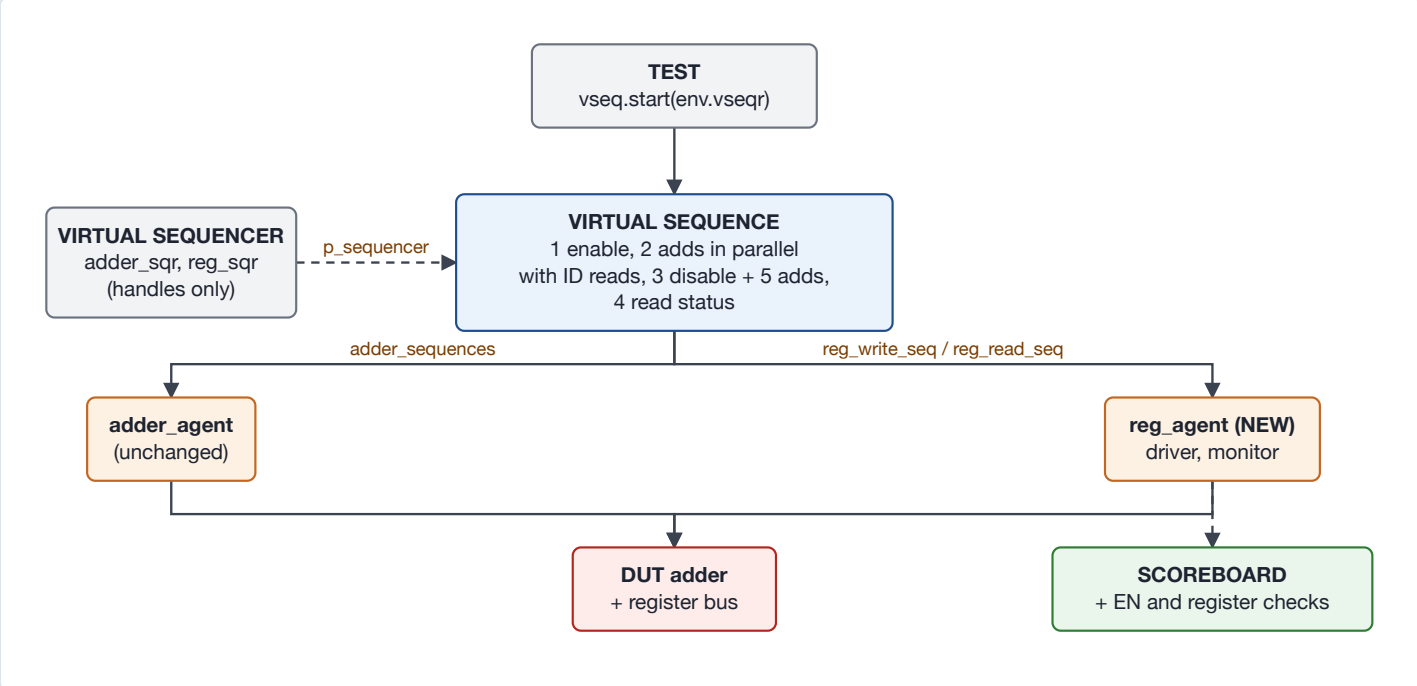
YOUR UVM REPO · ADDER_VIRTUAL_SEQ

ADDER → ADDER_VIRTUAL_SEQ: What Changes for a Virtual Sequence

Every file compared with ADDER · the DUT's new registers · the register-bus agent · virtual sequencer and virtual sequence · build / connect / run phase by phase · results

1. What this folder is: ADDER + a virtual sequence

`ADDER_VIRTUAL_SEQ` is your ADDER testbench with **one new idea**: a **virtual sequence** that runs **one scenario across two agents** — the adder agent and a register-bus agent — in a fixed order and partly in parallel. There is **no RAL** here: the register accesses are plain sequences with an address and data, so you see the virtual-sequence changes alone. Changed and new lines are tagged `// [REG]` (the DUT now has registers) or `// [VSEQ]` (for the virtual sequence).



Every file compared with ADDER:

File	Status	Why
`adder_item.sv`, `adder_sequences.sv`, `adder_sequencer.sv`, `adder_driver.sv`, `adder_monitor.sv`, `adder_agent.sv`, `adder_agent_cfg.sv`, `adder_coverage.sv`	**unchanged**	the adder sequence is reused as-is — the virtual sequence just *starts* it
`adder.sv`, `adder_if.sv`, `adder_scoreboard.sv`, `testbench.sv`	changed `[REG]`	same as ADDER_RAL: registers, enable-aware assertion, register checks, second interface
`adder_env.sv`	changed (+8 −1)	register agent + **virtual sequencer** and its handles
`adder_tests.sv`	changed (+5 −10)	new test `adder_vseq_test`: only starts the virtual sequence
`adder_pkg.sv`	changed (+8)	includes the new files
`reg_if.sv`, `reg_item.sv`, `reg_sequencer.sv`, `reg_driver.sv`, `reg_monitor.sv`, `reg_agent.sv`	**new** `[REG]`	the register-bus agent
`reg_sequences.sv`	**new** `[VSEQ]`	`reg_write_seq`, `reg_read_seq`: one access each, address and data given by you
`adder_vseqr.sv`, `adder_vseq.sv`	**new** `[VSEQ]`	the virtual sequencer and the virtual sequence

2. Step 0 — the DUT needs registers (same change in both folders)

A register map needs **registers**, and the plain adder has none. So the adder gets a tiny **register bus** and four registers. The adding logic itself is unchanged except for one word: it only adds when **EN** is 1.

Address	Register	Access	Meaning
0x0	`CTRL`	RW	bit 0 = **EN**: 1 = add, 0 = ignore inputs (reset 0 — **the test must switch it on**)
0x4	`ADD_CNT`	RO	how many additions were done (hardware counts by itself)
0x8	`LAST_SUM`	RO	the last sum
0xC	`ID`	RO	always `0xADD0` — a constant, handy to check the bus works

The bus (one access at a time): the testbench raises `reg_valid` with `reg_wr`, `reg_addr`, `reg_wdata` and holds them; the DUT answers with a one-cycle `reg_ready`, and for a read puts the value on `reg_rdata` in that same cycle.

What changed in `adder.sv` (lines tagged `// [REG]`):

```
// new ports
input  logic reg_valid, reg_wr;  input logic [3:0] reg_addr;  input logic [31:0] reg_wdata;
output logic reg_ready;          output logic [31:0] reg_rdata;

// the ONLY change to the adding logic: "&& en"
out_valid <= in_valid && en;
if (in_valid && en) begin sum <= sum_next; carry <= sum_next[N]; end

// new: the register block
reg_ready <= reg_valid && !reg_ready;                      // answer one cycle after the request
if (reg_valid && !reg_ready) begin
  if (reg_wr && reg_addr == 4'h0) en <= reg_wdata[0];      // write CTRL
  case (reg_addr)                                          // read any register
    4'h0: reg_rdata <= {31'b0, en};
    4'h4: reg_rdata <= {16'b0, add_cnt};
    4'h8: reg_rdata <= 32'(sum);
    4'hC: reg_rdata <= 32'h0000_ADD0;
  endcase
end
if (out_valid) add_cnt <= add_cnt + 1;                     // count every result
```

A lesson that came out of the run: ADDER's assertion `a_latency` said "every input produces an output next cycle" (`in_valid |=> out_valid`). With an enable bit that is **no longer true** — with EN = 0 the DUT must ignore the input. The virtual-sequence test (which disables the adder on purpose) made it fail. **When you add a feature, old checks may need updating**: `adder_if.sv` now reads `in_valid && en |=> out_valid` , and `tb_top` connects the DUT's `en` into the interface (`assign vif.en = dut.en;`) so the assertion can see it.

3. Step 1 — a register-bus agent (same in both folders)

The register bus is a **second interface**, so it gets its own agent — the same shape as `adder_agent` , with its own item, driver, monitor and sequencer.

New file	What it does	Same idea as in ADDER
`reg_if.sv`	wires + clocking blocks + 2 assertions (request held until ready; ready is a 1-cycle pulse)	`adder_if.sv`
`reg_item.sv`	one access: `wr` , `addr` , `data` (read data filled in by the driver)	`adder_item.sv`
`reg_sequencer.sv`	sequencer for register items	`adder_sequencer.sv`
`reg_driver.sv`	drives one access, holds `reg_valid` until `reg_valid && reg_ready` , copies `reg_rdata` into the item for reads	`adder_driver.sv`
`reg_monitor.sv`	one `reg_item` per completed access → analysis port	`adder_monitor.sv`
`reg_agent.sv`	builds monitor (+ driver, sequencer when active), connects them	`adder_agent.sv`

Scoreboard changes (`adder_scoreboard.sv` , tagged `[REG]`) — it now has a **third** input:

```
`uvm_analysis_imp_decl(_reg)                                    // third write function: write_reg()
uvm_analysis_imp_reg #(reg_item, adder_scoreboard) imp_reg;
bit en;                                                         // what the scoreboard believes
CTRL.EN is

function void write_in(adder_item txn);
  if (!en) begin n_ignored++; return; end                       // disabled: expect NO result
  ...
function void write_reg(reg_item t);
  if (t.wr) begin if (t.addr == 4'h0) en = t.data[0]; return; end   // writes tell us the mode
  case (t.addr)                                                      // reads are checked
    4'h4: check_reg("ADD_CNT",  t.data, n_out);
    4'h8: check_reg("LAST_SUM", t.data, last_sum);
    4'hC: check_reg("ID",       t.data, 32'h0000_ADD0);
  endcase
endfunction
```

Other small changes: `testbench.sv` creates `reg_if`, connects its six wires to the DUT and puts it in `config_db` for `uvm_test_top.env.reg_ag.*`; in `adder_env.sv` the config is now published to `"*"` instead of `"ag"`, because the new `reg_ag` needs it too.

4. Step 2 — plain register sequences (`reg_sequences.sv`, new)

Without RAL you say the address and data yourself:

```
class reg_write_seq extends uvm_sequence #(reg_item);
  bit [3:0] addr;  bit [31:0] data;
  task body;
    req = reg_item::type_id::create("req");
    start_item(req);
    if (!req.randomize() with { wr == 1; addr == local::addr; data == local::data; }) `uvm_fatal(...)
    finish_item(req);
  endtask
endclass
// reg_read_seq: same with wr == 0; after finish_item, data = req.data (the driver filled it in)
```

`local::addr` means "the sequence's `addr`", not the item's — inside `with { }` a bare name refers to the item.

5. Step 3 — the virtual sequencer (`adder_vseqr.sv`, new) and the env

```
class adder_vseqr extends uvm_sequencer;          // no item type: it never drives anything
  adder_sequencer adder_sqr;                      // just two HANDLES to the real sequencers
  reg_sequencer   reg_sqr;
endclass
```

Phase	New lines in `adder_env.sv`	Why
build	`reg_ag = reg_agent::type_id::create("reg_ag", this);`	second interface, second agent
build	`vseqr = adder_vseqr::type_id::create("vseqr", this);`	the virtual sequencer is a component in the tree
connect	`vseqr.adder_sqr = ag.sqr; vseqr.reg_sqr = reg_ag.sqr;`	give it the real sequencers — they exist only after build
connect	`reg_ag.ap.connect(sb.imp_reg);`	the scoreboard sees register traffic

6. Step 4 — the virtual sequence (`adder_vseq.sv`, new)

```
class adder_vseq extends uvm_sequence;                    // no item type
  `uvm_declare_p_sequencer(adder_vseqr)                   // typed handle: p_sequencer.adder_sqr /
.reg_sqr
  task body;
    wr.addr = 4'h0;  wr.data = 1;  wr.start(p_sequencer.reg_sqr);          // 1. enable
    fork
      begin add.num_items = 50; add.start(p_sequencer.adder_sqr); end      // 2. 50 additions ...
      repeat (5) begin rd.addr = 4'hC; rd.start(p_sequencer.reg_sqr); #200ns; end   // ... while
reading ID
    join
    wr.data = 0;  wr.start(p_sequencer.reg_sqr);                           // 3. disable
    add.num_items = 5;  add.start(p_sequencer.adder_sqr);                  //    these 5 must be
IGNORED
    rd.addr = 4'h4;  rd.start(p_sequencer.reg_sqr);                        // 4. ADD_CNT must be 50
    rd.addr = 4'h8;  rd.start(p_sequencer.reg_sqr);                        //    LAST_SUM
  endtask
endclass
```

Why a virtual sequence and not just a longer test? The scenario needs **order** across two interfaces (enable *before* traffic, disable *before* the last 5) and **parallelism** (ID reads *during* traffic). A normal sequence runs on one sequencer only. Putting the scenario in a virtual sequence also makes it **reusable**: another test, or a chip-level testbench, can start the same `adder_vseq` on its own virtual sequencer.

7. Step 5 — the test (`adder_tests.sv`, changed)

ADDER's test knew the adder agent's sequencer (`seq.start(env.ag.sqr)`). The new test knows **only the virtual sequencer**:

```
vseq = adder_vseq::type_id::create("vseq");
vseq.start(env.vseqr);
```

Phase by phase — what's new compared with ADDER:

Phase	ADDER	ADDER_VIRTUAL_SEQ adds
build	test → env → adder agent, scoreboard, coverage	+ register agent, virtual sequencer
connect	monitor → scoreboard, coverage	+ virtual sequencer's handles; register monitor → scoreboard
run	test starts `adder_sequences` on `env.ag.sqr`	test starts `adder_vseq` on `env.vseqr`; the virtual sequence starts everything else
check / report	leftovers, pass/fail	+ ignored inputs and register checks counted

8. Results and how to run

Step of the scenario	Result
1. enable	CTRL = 1 written
2. 50 additions **in parallel with** 5 ID reads	50 / 50 correct; 5 × `ID read add0 OK` at different times during the traffic
3. disable + 5 additions	**5 ignored** (`ignored(EN=0)=5`), no output from the DUT, no assertion failure
4. status	`ADD_CNT = 50` OK, `LAST_SUM` OK — 7 register checks, 0 errors

Run: `./run.sh` → `results/sim.log`, `results/dump.vcd`. In the waves you can see `reg_valid` pulses on the register bus **while** `in_valid` pulses on the adder bus — that's the `fork`.

The same scenario with RAL would replace `reg_write_seq` / `reg_read_seq` with `regmodel.CTRL.write(...)` / `regmodel.ADD_CNT.read(...)` inside the virtual sequence — combine this folder with `ADDER_RAL` (your UART_RX folder does exactly that).